The 4-Bit Image in a 32-Bit World

What if "emergent" AI abilities were there all along — we just couldn't see them?

There's a story that's been told so many times in AI that it's become background radiation. You've heard it: as language models get bigger, they spontaneously develop new abilities. Multi-step reasoning. Instruction following. In-context learning. These are the "emergent abilities" of Wei et al. (2022), defined as abilities not present in smaller models but present in larger ones — they appear suddenly past certain scale thresholds, like a phase transition in physics. The small model can't do them. The big model can. Something new was born.

This framing has real consequences. It's used to justify billion-dollar training runs. It shapes safety concerns — if capabilities can pop up unpredictably, how do you know what your model will be able to do next week? It even shows up in policy discussions about compute governance.

But what if the story is backward? What if the structure those capabilities run on was in the training data all along, and a small model already holds a blurred view of it? A model at any scale is a lens pointed at its corpus; parameters and training buy focus, not content. What "emerges" at scale, on this view, isn't a capability. It's our ability to read one that was already being transmitted. And the blur isn't uniform. Some of the structure comes into focus before the rest, in an order a smaller model's own outputs already carry, so you can forecast what a bigger model will come to do next from what the smaller one occasionally does now. Emergence has an order, and smaller models already know it.

I think the field has been looking at emergence through the wrong lens. Let me offer you a different one.

Here's the result that now anchors it, along both axes of scale. Standard evaluation only checks a model's single most likely output (greedy decoding, or argmax at temperature 0), where a one-billion-parameter model usually emits a wrong answer and scores zero. But if you instead draw 64 to 256 random temperature-1 completions per question — reading the lower-probability *sampled tail* of the model's output distribution — you can count how often the right answer appears among those non-greedy draws. Hold the training data completely fixed first: at 300 billion tokens, where every Pythia model has seen the exact same text in the exact same order, how often a 1B model hits the answer in its sampled tail already orders which specific questions ("items") larger models (1.4B, 2.8B, 6.9B and 12B) solve under greedy decoding (rank concordance .218 for Pythia-1B and .278 for cross-family OLMo-2 1B on a −1 to +1 scale where 0 is chance, both p = 1e-4, including .194 and .270 on items where both 410M and 1B score zero at greedy decoding), while a linear classifier trained on the 1B model's internal hidden-state activations (a *linear probe*) reads nothing (.026). Then let training run: on fresh versions of the field's own emergent tasks, the sampled outputs of those same two 1B models forecast which items come into focus first across all four outcome families I preregistered (SmolLM3-3B, OLMo-2 7B, OLMo-2 13B and Comma, all eight tests p = 1e-4). And when you look inside the 1B blur to see what it's actually resolving, you find coarse-to-fine resolution in stages: the small model resolves the coarse outline of the answer space first, and larger models resolve the fine counterfactual rule on top of it. It's a forecast of item order on these task renderings, not of which tasks a model will clear, and it doesn't by itself tell us how the model learns them.

One thing to say before the analogy. This is the second version of that claim. The first, which most of what follows was written under, put the from-below signal in the model's hidden state, treated pre-training loss

as the currency the focus is bought in, and carved out an exception on the strength of a formal-language result. Twenty preregistered experiments later I've moved all three, and I'd rather say so in one place than let the framing drift to fit the results. The core thought survived them. The mechanism I'd hung on it didn't, in three places, and the section after the analogy names them.

The 4-Bit Photograph

Imagine I hand you a digital photo of a cat. The catch: it's saved at 4 bits per color channel. That means each pixel can only be one of 16 shades of red, 16 shades of green, 16 shades of blue. The image — if we're being generous — is *blocky*. The fur looks like it was painted by a very drunk pointillist. The whiskers blend into the background. The eyes are two dark smudges.

Now imagine I show you the same photo at 32 bits per channel. Suddenly there's detail. You can see individual whiskers. The texture of the fur. The catchlight in the iris. A perfectly recognizable cat.

Did the cat "emerge" when we went from 4-bit to 32-bit? No. A classifier looking at the 4-bit version fails not because there's no cat signal in the image, but because the quantization noise swamps that signal at the feature boundaries. The evidence of the cat is present; the detector can't separate it from the noise.

Now, I should flag where this analogy breaks — because it breaks in a way that actually strengthens the argument. Quantization is lossy. You genuinely cannot reconstruct the individual whiskers from the 4-bit file; that information is destroyed, not hidden. If small models were like a 4-bit image in this strict sense, probing their internals would turn up nothing, and no amount of re-sampling would ever produce the missing capability. But as we'll see, that's the opposite of what the evidence shows. The information in small models *is* recoverable. So a small model isn't a lossy compression of a big one. It's closer to a noisy channel carrying the same signal at lower power: the structure is on the wire, but it sits too close to the noise floor for a standard receiver to decode.

Keep both pictures in mind. The 4-bit cat is the intuition pump. The noisy channel is the claim — and the channel is longer than it looks. It runs from the training distribution, through the weights, to the decoded output, and every phenomenon in this essay is a signal-to-noise story at one of those three stages.

Resolution, Not Discovery

Call the operative quantity resolution. A model trained on next-token prediction is forced to model whatever structure the corpus has, syntax, reasoning chains, world knowledge, long-range dependencies, all of it, because that's what predicts the next token. None of that structure is invented by the model. It's in the distribution, put there by the people who wrote the text, and it exists upstream of any particular model. What a model has is a view of it at some fidelity, and fidelity is bought two ways: with parameters, which set how finely the model can carve its representational space, and with training, which spends that capacity. A 7B model at the end of its run and a 70B model a fifth of the way through its own are both views of the same scene, and neither is the scene.

The blur isn't uniform, and that's the part of the picture that does the work in this essay. Some structures come into focus early and some late, and the order isn't arbitrary. It's set by how much fidelity each structure needs before it stands out from the alternatives that compete with it, which is a property of the structure and the corpus more than of the model. That makes it a claim you can test without waiting for the

bigger model, because a model that's still blurry on a task isn't silent about it. Its output distribution puts a little more mass on the right answer for the items nearest to coming into focus and a little less on the rest, and that gradient is readable by sampling long before greedy decoding produces the answer once. When I say "forecastable from the blur" below, that's what I mean, and it's the form of the claim that has held.

The familiar picture is still the right intuition. A small model "knows" the first step of a multi-step chain but can't hold a full chain together; the correct continuation is present but competing for bandwidth with a dozen plausible ones, and sampling noise, temperature or a stray activation tips it. At higher fidelity the boundaries sharpen and the right path stands out. The model appears to have gained the ability to reason. The ability was latent in the smaller model; what changed is whether it survived decoding.

Now the three revisions. First, the layer. The first version of this essay put the from-below signal in the hidden state: a linear probe finds the answer in a small model's activations before the model can produce it. Within a task that's true, and I've measured it with the same weights on two task classes. But asked to forecast which items come into focus first, the probe read nothing on the two training runs where it competed, and the sampled tail read the order on those two and on six more, every one of them a run nobody had looked at. The representation isn't the leading indicator. The sampled tail of what the model already says — its hit rate across repeated temperature-1 draws when its top greedy answer is still wrong — is.

Second, the knob. I used to say the bits in this picture are bought with loss reduction, so pre-training loss is the currency and parameter count just one way to earn it. Loss is a gauge of fidelity, and a better one than parameters, but it isn't fidelity. Read Pythia's larger models at the checkpoints where their held-out loss equals a smaller model's final loss and the capability profiles differ by more than the larger run's wobble between neighbouring checkpoints, with no direction at all, and the smaller model's own final, which the design took at face value, moves by dozens of items over its last thousand steps. The knob is fidelity, bought by parameters and by training, and loss is an imperfect reading of where it sits.

Third, the carve-out. There is a class of capability that genuinely doesn't exist below a threshold, because the connected structure it runs on hasn't been assembled yet, and I take that class seriously in the section on objections. But I've established it only on synthetic data, where the ground truth is known by construction. No real-model capability has been shown to belong to it, and the one I looked at hardest, string reversal, doesn't.

That's the framing. Here's why I think it's more than a nice picture: it maps onto real results, and the ones that moved it are mine.

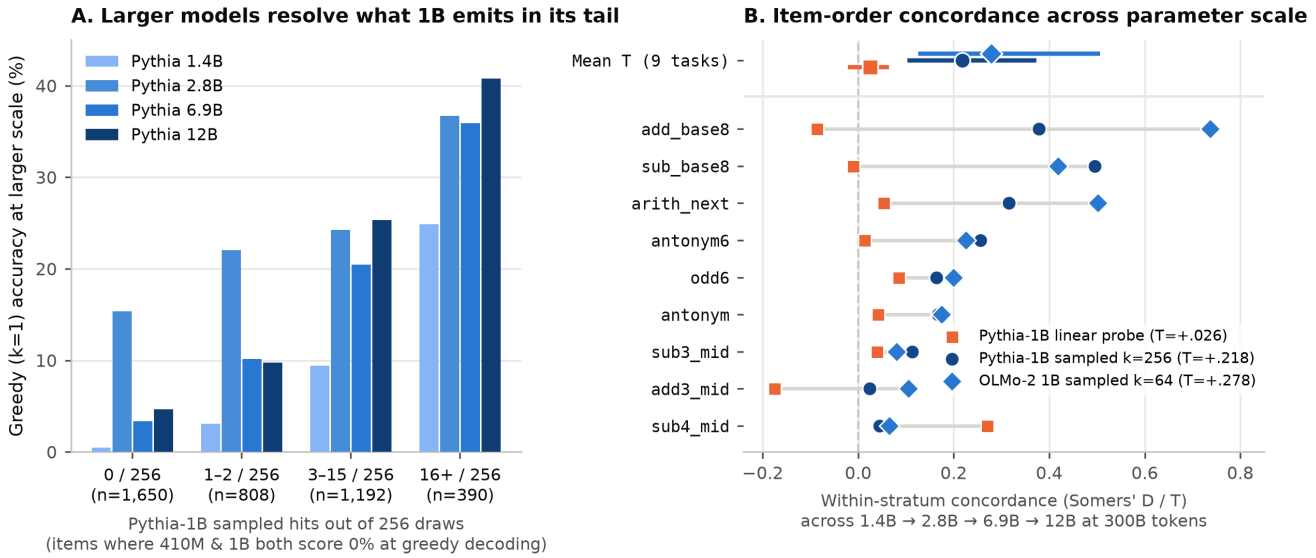


Figure 1. Resolution along the parameter ladder at fixed training data (300B tokens, step 143,000). A: Restricting strictly to the 4,040 items across the nine informative tasks where both Pythia-410M and Pythia-1B score 0% under greedy decoding, items emitted more often in Pythia-1B's 256-draw sampled tail are systematically the ones that resolve under greedy decoding at 1.4B, 2.8B, 6.9B and 12B (reaching 40.8% greedy accuracy at 12B for items with 16+ hits at 1B, versus 4.7% for items with 0 hits). B: Within-difficulty-stratum rank concordance (Somers' D and its stratum-weighted mean T, ranging from −1 to +1 with 0 at chance, shown with 95% family-bootstrap intervals) ordering item resolution across 1.4B, 2.8B, 6.9B and 12B at 300B tokens. Both 1B sampled-output predictors carry the parameter-ladder order (Pythia-1B k=256 T = +.218, OLMo-2 1B k=64 T = +.278, both p = 1e-4), while the Pythia-1B hidden-state linear probe reads nothing (T = +.026, p = .13).

What the Evidence Actually Shows

Hidden states don't lie. The probing literature has shown, repeatedly, that a model's activations can encode information its outputs don't reliably reflect — the state of an entity through a narrative, the truth value of a claim, an intermediate quantity in a calculation — recoverable with nothing fancier than a linear probe, even when the generated answer gets it wrong. The tuned lens (Belrose et al., 2023) gives the cleanest window into the mechanics: decode each intermediate layer directly and you find refined predictions forming well before the output layer, predictions the final decode doesn't always honor. The model encodes more than it surfaces. This is the result that rules out the strict lossy-compression picture: if the information were destroyed, the probes would find nothing.

Induction heads at any size. The attention pattern that enables in-context learning — the "induction head" that copies the pattern A…B → A…B — has been found in models as small as 125M parameters (Olsson et al., 2022). But in small models, the induction signal is weak. It's competing with other attention patterns and often gets overwritten. In large models, it dominates. The capability (in-context copying) exists at every scale. What changes is whether it wins the competition for representational resources.

The metric that changed the debate. Schaeffer et al. published a paper in 2023 that kicked up a lot of dust. They showed that if you swap out binary evaluation metrics (exact match: did the model output

exactly the right answer?) for continuous ones (Brier score, log-probability), the "sudden emergence" hockey-stick curves turn into smooth, predictable improvement curves. The field's response was divided: some said "see, emergence is fake," others said "the coarse metric can't detect real capability thresholds."

Under the resolution model, both are right. The binary metric is like trying to detect cat features in a 4-bit image: too coarse to register the partial signal present in the small model. The continuous metric gives you credit for having *more probability mass* on the right answer, even when your argmax still picks the wrong token. Neither metric says anything about whether the latent structure exists — they only measure how reliably the model can deploy it.

Sampling reveals the latent. Hu et al. (2023) took this to its logical extreme with a technique called PassUntil — essentially, sample from the small model hundreds or thousands of times until it generates a correct answer by chance. And it works. Small models *can* produce correct reasoning chains; they just do so unreliably. With enough samples, the capability becomes visible. The model passes the task at a resolution the standard metric can't see. This is the noisy-channel picture made literal: repeat the transmission enough times and the signal climbs out of the noise.

What's Happened Since the Mirage Debate

The 2023 metric-artifact argument didn't settle anything; it forked into several research programs, and the results since then mostly tighten the case for resolution over discovery — with one important exception I'll get to.

Emergence tracks loss, not parameter count. Du et al. (2024) trained models of different sizes to the same pre-training loss and found they perform the same on downstream tasks — and that "emergent" abilities appear when loss crosses a threshold, regardless of model size, and regardless of whether the metric is continuous or discontinuous. Read carefully, this cuts both ways. It weakens the pure metric-artifact position (the thresholds don't fully dissolve under continuous metrics), but it supports the resolution framing's core substitution: what matters isn't parameters, it's representational fidelity, and pre-training loss is a far better proxy for fidelity than parameter count. The "bits" in my metaphor are bought with loss reduction, and scale is just one way to pay for them. That was the first version's reading; Prediction 3 below is where I tested it on Pythia's checkpoints, and on that battery loss turned out not to be the whole currency.

Emergence became predictable. Ruan, Maddison, and Hashimoto (2024) built "observational" scaling laws from about a hundred public models and showed that performance across model families is a function of a low-dimensional capability space — and that several canonical emergent phenomena follow smooth, sigmoidal curves that are predictable from small models. If capabilities were genuinely discontinuous births, you couldn't forecast them from below the threshold. You can, at least across model families at the scales they measured; in my own runs from 1B, collapsing a battery into a 34-dot across-task ranking didn't work because 23 of the 34 tasks sit tied at zero at 1B, whereas forecasting within tasks at item grain (which individual test items resolve first) from the 1B sampled output tail worked across both parameter scale and five model families' training runs, as I show in the next two sections.

We now understand *why* downstream prediction seemed impossible. Schaeffer et al. (2024) followed up the Mirage paper by tracing exactly where predictability is destroyed: benchmark scores are

computed from log-likelihoods through a chain of transformations (normalization, comparison against incorrect choices, argmax) that progressively degrade the statistical relationship between performance and scale. This is the measurement side of the resolution story made mechanical — the capability signal degrades in the *evaluation pipeline*, not (only) in the model.

Inference compute substitutes for scale. Snell et al. (2024) showed that on problems where a small model has a non-trivial success rate, spending compute at test time (search, repeated sampling, verification) can outperform a model 14× larger. That's PassUntil turned from a measurement trick into an engineering strategy, and it's exactly what the resolution framing predicts: if the capability is latent but noisy, you can decode it by integrating over more samples instead of raising the transmitter power. The entire reasoning-model era — inference-time search amplifying capabilities that were latent in the base model — is this principle at industrial scale.

The field caught up to the ambiguity. Berti, Giorgi, and Kasneci (2025) surveyed the emergence literature and found the definitions themselves inconsistent — emergence variously pinned to scale, loss, task complexity, quantization, or prompting strategy. That's what you'd expect if the field has been using one word for two different things: the existence of a latent structure, and the reliability with which it can be decoded.

Unlearning keeps failing, and the failure is diagnostic. If a capability were something a model *has*, you could remove it. In practice, nobody can. Capabilities "unlearned" by state-of-the-art methods come back after fine-tuning on as few as ten unrelated examples (Łucki et al., 2024). Simply quantizing an unlearned model — no adversary, no retraining — restores most of the supposedly forgotten knowledge (Zhang et al., 2024). And linear probes recover the knowledge from the "unlearned" model's activations the whole time. The pattern is exactly what the resolution framing predicts: these interventions operate at decode time, lowering the probability of *expressing* the capability without touching the structure that carries it. You can doctor the print; the negative is intact. On the phase-transition view, it's not obvious why removal should be so much harder than acquisition. On the resolution view, it's obvious: the structure was never the model's to lose.

Lenses pointed at the same scene converge. Huh et al. (2024) documented that as models scale, their internal representations converge — across architectures, across training runs, and even across modalities, with vision models and language models arriving at increasingly aligned representational geometry. If each model constructed its capabilities, there would be no reason for independent constructions to agree. There's an obvious reason if capabilities are resolved rather than constructed: many lenses, one scene. In the first version of this essay that was the strongest evidence that the latent structure lives upstream of any model, in the distribution they're all pointed at. It isn't any more. The cross-family forecasts in the next section carry that argument now, and what happened when I measured convergence at the level of individual tasks ("task grain") is next.

I tried to measure that claim at task grain, and the measurement didn't survive its own calibration. The four training runs this program had already scored, Pythia 2.8B, OLMo-2 7B, SmolLM3-3B and Comma, 92 checkpoints between them, gave me the test Huh et al.'s result doesn't by itself supply: for each task, does the agreement between a model's representation of that task's 500 items and three released models from the other families run ahead of the model's general convergence trend before the task first performs? The outcome was known going in, so this was a preregistered reading, not a forecast. On

fifteen tasks across the four runs, .61 of the task-specific agreement present at the end of training was in place before the "first clear" (the first training checkpoint where greedy accuracy rises significantly above the random-guessing floor), the sign consistent across tasks at p = 5e-4 under an exact sign-flip test. I reported that with a bound on it, because the eligibility rule pushes the null to about .5 and the never-performing ("flat") tasks' checkpoint-to-checkpoint scatter ran past what my calibration covered. Then I built the null the bound was asking for. I put the tasks that never perform through the identical measurement, same selection rule, same clear points, ten thousand batteries, with every input committed and known to me before it ran, so the null was the only thing I hadn't seen. A quarter of those placebo batteries reach .61 (p = .25). The lead over that null is .17 with an interval from −.26 to +.70, which covers zero, and no single run separates by itself. OLMo-2 7B is the one that stings: it looked strongest at .96, and its own flat tasks produce .84 under the same selection. The decision rule I'd preregistered fires on one null battery in nine. So the task-specific lead did not separate from the flat tasks' drift, and what survives is narrower. The agreement did not arrive with performance, since the sign-flip reading against that cell stands; whether it led the general drift is not distinguishable at this resolution. The lens claim goes back to resting on Huh et al.'s global result, with this disclosed beside it. Then I ran the test once more, on two training runs nobody had read: Pythia 6.9B and OLMo-2 13B, forty checkpoints between them, with a rank statistic that needs no excess and no eligibility screen, each rising task's pre-clear agreement growth ranked among every never-performing task's at the same checkpoint, so the null mean is exactly one half and owes nothing to the data. The statistic was sealed before either run was touched and declared underpowered in advance against the shape the four known runs had shown. The rising tasks ranked at the flat pool's median: U = .45 on twenty-six cells, interval .33 to .61, p = .73. The shape that had motivated the design, the option and string tasks leading, did not carry; the three families that led on the known runs sit below one half on the sealed ones. The two runs disagree in sign, Pythia 6.9B above one half and OLMo-2 13B below, which the pooled test cannot resolve and the preregistered claim rule ("licence") does not let me name beyond saying so. One family, sequence extrapolation, outran the flat pool on both runs. The lens claim stays a citation, and this is not evidence of absence: the resolution would have missed the known runs' effect three times in four.

The exception. Lubana et al. (2024) is the most serious recent evidence on the other side, and I'll take it up in the next section.

What I Found When I Tested It

The citations above are other people's results read through my framing, and that's cheap. So over the past three months I ran the framing against its own predictions on the smallest real models where the test is still affordable, with every pass/fail criterion and allowed conclusion ("claim licence") frozen in a Git commit and tagged before the data existed, pre-analysis integrity checks ("gates") that halt the run if a control fails, the analysis run once, and a projected outcome sealed before each analysis so I couldn't grade myself on a curve afterwards. Twenty experiments closed under that protocol. Three never reached a verdict, two because their untrained control fired before any scored model was touched and one because its frozen analysis crashed on an input it should have refused; one failed on a criterion I'd misspecified and had to be re-run as a fresh preregistration, and the sealed projections missed about as often as they hit, which is the point of sealing them. The rest is what follows. All of it, preregistrations, frozen analyses, every raw draw

and the synthetic calibration runs, is public at github.com/lxman/decodable-is-not-learned and archived at DOI 10.5281/zenodo.21830421.

Start with the instrument. The three-signature test — checking whether a capability shows (1) linear-probe decodability in hidden states, (2) a non-zero hit rate under deep temperature-1 sampling, and (3) item-level forecastability before greedy emergence — is only worth anything if it separates the two classes where class membership is known by construction, so I built both in miniature: transformers of one and ten million parameters trained on a Lubana-style formal language, where the capability provably doesn't exist below the percolation threshold, and on a modular-arithmetic grokking task, where latent structure demonstrably strengthens before the model can use it. Then I asked the probe leg of the test to tell them apart, with one correction my first attempt hadn't made: a model-by-task evaluation pair (a "cell") counts as present only if its probe beats the same probe trained on an untrained copy of the network with the same random initialization — its *untrained twin*. That correction matters more than it sounds. Nine of the thirty untrained twins passed the raw criterion, and whether they did was a property of the data split rather than the network. The grokking split hands a random network a held-out sample of the same pairs and it decodes them fine; the Lubana split holds out whole entities and the random network reads nothing. With the floor subtracted, the discriminator separated the classes: grokking before its transition present in 9 of 10 cells, Lubana above threshold 10 of 10, Lubana below threshold 0 of 10. A second experiment looked harder at the below-threshold side, sweeping graph density from a quarter of critical to .85 of it across forty trained networks and 320 probe sites, and found nothing decodable anywhere: zero sites fired, against zero of 480 untrained sites, with the capability metric flat at chance. That rules out sub-critical structure accumulating to 39% or more of the above-threshold margin, and no finer; the observed reading was 4%, inside the blind region, so the carve-out survives at the resolution I could buy and is untested below it. All of this is synthetic, ten million parameters and under, and it certifies the probe leg only. It's the calibration, not the result.

On real models the instrument broke twice before it worked, and the way it broke became a methods paper. A linear probe on Pythia with a couple of thousand items and a thousand-plus features will decode almost any label from an untrained network, because a random high-dimensional expansion of the token sequence is a reservoir that carries the prompt's surface statistics. My first battery fired its untrained control on every capability. The rebuilt one, with splits designed so that a lookup table scores chance, still leaked surface statistics in 13 of 25. Decodable is not learned, and the untrained twin is the only control that tells you which one you've got.

The third version screened every task against that twin before admitting it and ran the probe ladder properly: 34 tasks in 16 families, probes at 410M and 1B ranking them, greedy decoding at 2.8B, 6.9B and 12B scoring the outcome, with a lock so the outcome couldn't be seen before the ranking was on the record. It failed, and failed uninterpretably: rank correlation .37 with an interval from -.18 to .76, because 22 of the 34 probe margins were tied at zero and there was almost nothing to rank. Two things outlived the omnibus test. The string-reversal tasks carried the second and third highest probe margins in the battery and scored exactly 0.000 at greedy decoding at every eval size. And the projection's named disconfirmer fired: two arithmetic tasks the probe had read as silent rose above their chance floors by 2.8B, which is what a percolation-class capability would look like if there were one in the battery. Both became experiments of their own.

Reversal first, because it's the cleanest version of the question. On the same 410M and 1B weights where a linear probe reads the answer's first character from the residual stream (the transformer's main layer-to-layer hidden state) at margins of .57 to .77 over the untrained floor, what does the output path do with that character? Greedy decoding emits it at the floor, .026 to .052 against critical values near .09, in all four cells, while a copy-control task on the same harness emits its first character at .994, so the instrument sees emission wherever emission exists. That's the famous zero with the units matched (one character read, one character emitted) and the scale matched (same weights), and matching them surfaced nothing. It was never a Schaeffer metric cliff. Then I asked whether the information was in the output distribution at all. Pure temperature-1 sampling, 128,000 draws per cell, produced one complete correct reversal: one draw, on the shortest strings, at 1B. The other three cells were walls, which I can only report as bounds, about 2e-5 per draw at that budget. Then I tripled the budget under a fresh preregistration and let a frozen analysis adjudicate the deeper look. The one draw became a rate, ten verified reversals in 512,000 draws, so the one-in-128,000 event was a channel and not a fluke. And the 410M wall fell: zero at 128,000 draws, three in 512,000, all on four-letter strings and about nine times what blind letter-guessing would produce. Nothing about the model changed between those two measurements; only the budget did. Plot pass@k (the probability of getting at least one correct answer in k samples) against k and you get the textbook emergence jump, reproduced on demand under preregistration, with the cause sitting entirely on the instrument side.

The third signature I can report as a forecast rather than a story. Forecastability is the easiest signature to fake after the fact, since once you've seen which items a model got right some account of why those items is always available, so I froze the account first: a structural functional of the answer string, computable from the item file with no weights, no forward pass and no samples, its value on all 500 items committed, and then 1.15 million fresh draws to find out whether it had picked the winners in advance. It had. Seven of the eight items that fired came from the 45-item class whose answers repeat a character, where chance predicts 1.9 ($p = 1.6e\text{-}4$), a twenty-three-fold difference in per-item rate, and six of the eight had never fired in any earlier experiment, so this isn't the same easy items showing up again. A capability that's simply absent has no reason to fail non-randomly. A graded quantity being read through a hard threshold has exactly that reason.

I should be careful about what that doesn't settle. Answers that repeat a character are cheap in a second sense: they're more probable strings to begin with, they overlap more with the input the model is copying from, and this model copies almost perfectly. The functional might have been forecasting which answers are cheap to *emit* rather than which ones the model half-knows how to *reverse*. So I ran the experiment that splits those. Within the forty-five repeat-character items, thirty-two have a reverse that's one copy error from the input (swap two letters, or start one position off and copy forward) and thirteen don't, at identical entropy. If the successes were copy errors that happened to land on the right string, the thirteen should stay close to silent. They didn't. In 368,640 fresh draws, four of the thirteen produced correct reversals (`qsee` to `eesq`, `ddfv` to `vfdd` twice, `eewc` to `cwee`, `dwtt` to `ttwd`) against thirteen of the thirty-two on the other side, which is what no effect looks like ($p = .40$, test frozen in advance). The reachable items do fire at a higher per-draw rate, but four hot items carry most of that, the design had declared itself underpowered against a partial effect before it ran, and the honest statement is that the non-reachable rate is at most .42 of the reachable one. Scoring the same draws against the other one-edit strings that start with the right letter shows what the channel near these items is: it gets the first character right and then garbles.

On some items the model's favourite output is the last letter followed by the input copied forward (`cara` gives `acar` twenty-two times and the reverse once); on others it swaps the last two letters. The exact reverse is one misfire among several, neither privileged nor absent, and the repeat-character items with no copy route still fire at about four and a half times the rate of the all-distinct ones. The cheapness belongs to the string, not to a copying artifact.

There's one more thing the measured distributions say that the clean version of my own framing got wrong. I'd been picturing the output distribution as a blurred image of the capability: lower the detection threshold and the right answer brightens smoothly out of the noise. It doesn't. At the first output position, the bulk of the letter mass tracks shallow lexical statistics, which characters appeared in the prompt with the earliest weighted hardest, and by that ranking the correct character loses to the input's interior on about four items in five. The capability doesn't sit just under the surface of the bulk; it sits below a foreground of copying statistics that dominates at every scale I measured. That's why the instruments disagree so sharply. Probes and deep sampling can separate signal from that foreground; argmax and bulk probability mass can't. The noise in the noisy-channel picture isn't white. It's the model's own shallower habits, and any instrument that doesn't cancel them will read format where you hoped to read capability.

Reversal is one task, and a peculiar one, since the answer's characters sit in the prompt. So I went back to the two arithmetic tasks from the 34-task battery, the ones the probe had read as silent and the disconfirmer had named, and put all three instruments on one label per task, one chance floor and one bar, on the same weights: a probe trained on the committed activations and read on the 500 eval items, temperature-1 sampling at 32,000 draws, and greedy decoding, all scored on the middle digit of a three-digit subtraction and on the last digit of the next term in an arithmetic sequence. The probe reads the digit in all four cells, .23 to .27 against bars of .17 to .19, with the untrained twins at .04 to .10. Sampling and greedy decoding read it in one cell each, the sequence task at 410M, and nowhere else. At 1B the sampler is below chance on that last digit, at .10, while the residual stream of the same forward passes carries it at .27: a model that's wrong about a + 4d is usually off by one step, which changes the last digit, and the representation holds the digit the sampler doesn't emit. Presence before performability, four cells of four, on a task class where the answer isn't in the prompt. And the earlier silence was mine, not the model's. On the sequence task, the same 1B activations that read the last digit at .27 read the label I'd originally given the probe, a residue mod 7 chosen to defeat lookup and validated on held-out basis values, at .12 against a .21 bar. The target is part of the instrument. A probe's silence is a statement about a label and a split until the same activations have been read against a label the representation could carry.

What hasn't worked is the claim I most wanted: that you can rank tasks from below. I've now tested it three ways on that battery, with the outcome scored at 2.8B, 6.9B and 12B. The probe ranking was the uninterpretable .37 above. Swapping the instrument for the sampler, with a chance floor that gives format-guessing no credit (the majority answer, or one of n listed options, applied to both sides), didn't help: ten of the eleven tasks that clear the floor by 12B sit below it under sampling at both small sizes, with bounds that exclude the floor, so the predictor was one positive task against 33 zeros (AUC .55, the interval touching .5). Taking the threshold away and scoring the same rates as log-excess over the floor ranks the battery at .61, the probe's own number, with an interval from .37 to .89, and the floor alone ranks it at .42. The preregistered reading is "not detected at this resolution," and I think the resolution is the problem in a specific way. At 1B, Pythia doesn't match a random option-copier on multiple-choice-shaped tasks under sampling or under greedy decoding, and 2.8B is where it crosses that line; whatever leads that crossing isn't

visible to a probe margin, a sampled rate or a greedy score at 1B the way I measured them. Ladder order within a task and forecasting across tasks are different claims. The first has held everywhere I've looked. The second hasn't been found, and I haven't shown it doesn't exist.

So I ran the version with nothing left to blame. The one outcome in this program nobody had seen was the inside of training: which of each task's 500 items Pythia-2.8B can emit at each of 21 intermediate checkpoints, something no one — including me — had ever measured. I committed a per-item probe reading at 410M and 1B, the matched-label instrument from the arithmetic test, the one that demonstrably reads presence, and tagged it before any checkpoint was ever loaded; then I swept the checkpoints and asked whether the items the small representation carries are the items training makes emittable first, holding each task's structural difficulty fixed — carries and borrows, option position, magnitude — so a shared prior couldn't manufacture the answer, with an untrained twin as a tripwire for surface statistics. Preregistered power: .95 to see an ordering effect of .15. The answer is no. The probe's rank concordance (Somers' D, on a −1 to +1 scale where 0 is chance) with emission order is −.02 stratified within difficulty tiers and .02 raw, null under every preregistered variant, at both probe sizes and in the 12B replication. And the same run carried a competitor I'd preregistered: the sampled rate per item — sixty-four temperature-1 draws from the 1B model, committed to the repository two experiments earlier — forecasts that same sealed order at .17 with p = 1e-4, through the same difficulty strata, and the probe adds nothing once you condition on it. A finding that arrives as a secondary isn't established, so I ran it again as the primary on the one outcome still unseen: 6.9B's 22 intermediate checkpoints, which nobody had queried either, with the same committed 1B counts as the predictor, the probe reading as the named competitor, and preregistered power .98 to see .15. The sampled rate forecasts 6.9B's emission order at .20 with p = 1e-4. The probe forecasts nothing again (−.02), adds nothing once the sampled rate is conditioned on, and the per-task concordances land in the same order at the top on both outcomes: base-8 subtraction, then the six-option antonym task, then base-8 addition. The 410M model's counts replicate at .14. What predicts which items training surfaces first is how often the smaller model already emits them, not what its residual stream linearly carries, and that now holds on three hidden outcomes — 2.8B's training, 6.9B's, and a second family's — though the third took its own lineage's predictor to see. Presence before performability keeps standing; the representation really does hold answers the model can't produce. But at item grain the ordering information lives in the output channel, and the resolution story as I'd stated it privileged the wrong layer. What I can't separate inside a sampled rate is reachability from difficulty I haven't named; the difficulty I did name is held fixed, and the next test was a predictor from a different model family, where shared difficulty survives as the null and shared training data doesn't. I ran it: OLMo-2, a family nobody in this program had ever queried — its 1B's sampled counts sealed before its 7B's main pre-training ("stage-1") checkpoints were touched, Pythia-1B's committed counts riding along as the cross-family test, both preregistered at the same bar. The lineage version fired again: OLMo-2 1B's counts forecast the order in which OLMo-2 7B's training makes items emittable at .22 with p = 1e-4, a third hidden outcome and a second family for the same shape of claim. The cross-family version did not. Pythia's counts carry real structure into OLMo's training order — .095, with zero of ten thousand permutations reaching it — but under the .10 bar I froze in advance, and the shortfall sits exactly where the preregistration said dilution would sit if it appeared: three digit-manipulation tasks Pythia-1B almost never emits at all, ten to thirty live items out of five hundred. So the sentence I get to keep is lineage-bound — the smaller model of the same lineage forecasts what training surfaces first — and the cross-family sentence stays unlicensed at that draw budget; at four times it, Pythia-1B's counts clear the same bar on the same outcome, .155, an outcome

already on disk, so a bar cleared and not a forecast made, and I come back to what that bought below. One number from the non-gating side reframes the question, though. Read backwards against outcomes already on the record — a reading, not a forecast — the richer OLMo predictor ranks Pythia's old training orders at .26 and .30, two to three times the strength with which Pythia's thin counts forecast OLMo's, so the family boundary looks cheaper than predictor sparsity. I tested both readings on the bytes already on disk, with the functionals and the tree frozen before anything was computed and everything else — the concordances included — disclosed as known. Thinning the richer predictor to the thinner one's draw budget, task by task, closes the reverse-direction gap entirely: at matched budgets OLMo's 1B ranks Pythia's old training orders no better than Pythia's own 1B did, so that asymmetry was information density, not the family, and half of the lineage advantage on OLMo's own outcome is density too. The mechanism half asks what the count is reading. Holding fixed, inside the difficulty strata, the answer's length, its repeated characters, its overlap with the prompt, and how often the small model says that answer when it is wrong, the forecast survives at .13 with zero of ten thousand permutations reaching it — three fifths of it — so what the count carries is not any of those four. What the other two fifths were is the more interesting number: length first, then the answer's prior, and almost all of it on base-8 addition, where a three-digit sum is the top carry; the option tasks lost nothing. And that wrong-answer frequency — the model's *habit prior* — turned out to forecast on its own: which number strings the 1B habitually emits as wrong answers ranks which of those numbers the 7B learns first, at .20, including on the digit tasks where the count is silent. That is a second mechanism sitting under the first, not a correction to it. So I bought the draws. Sampling Pythia-1B two hundred and fifty-six times per item instead of sixty-four — the committed sixty-four regenerated byte for byte as the gate on every item, three fresh seeds beside them, the instrument tagged before a draw was taken — its counts forecast OLMo-2 7B's training order at .155 with zero of ten thousand permutations reaching it, against the .095 that missed the bar: the shortfall was predictor density. The outcome was already on disk throughout, so that is a bar cleared, not a forecast made; the sealed cross-family forecast is the next experiment. Two things the extra draws showed that the verdict alone doesn't. The four independent sixty-four-draw blocks read .095, .108, .095 and .094 — the earlier miss was one block of four, and one of the four clears the bar on its own, so a bar decision that close was a coin flip at the time it was made. And at matched draw budgets OLMo's own 1B still ranks OLMo's order .05 ahead of Pythia's, the same margin as at sixty-four draws: density explains the shortfall below the bar and not the lineage gap, which doesn't move. Where the draws bought signal is telling too — on the tasks where the thin predictor had almost no live items, base-8 addition and a three-digit subtraction the sixty-four draws had barely touched, and nowhere on the antonym task, which was already dense. The across-task ranking stays unfound either way. That left the sealed version, and OLMo-2 ships one more model: its 13B line publishes stage-one checkpoints from a true step zero, and nobody — again including me — had ever queried them. I tagged both predictors where they already sat, Pythia-1B's two-hundred-fifty-six-draw counts and OLMo-2 1B's own, fixed the two tests at the same .10 bar, wrote the projection down, and swept sixteen grid points of 13B's training with the endpoint reproduced byte-for-byte through a second loader path as the gate. The cross-family forecast fired: .126, with zero of ten thousand permutations reaching it, on an outcome that did not exist on disk when any draw was taken. The lineage predictor fired beyond it at .18, so both components are real and the lineage one is still the larger. The ladder inside the result seals the density story: the four sixty-four-draw blocks read .072 to .080, every one under the bar, so at the old budget there is no cross-family forecast of 13B at all, and at four times the budget there is. And the per-task table finally says where the family boundary lives. The arithmetic tasks carry the cross-family forecast — base-8 addition at .46 —

while the antonym task comes in negative for Pythia's counts and .26 for OLMo's own, the first task where the two predictors disagree in sign: what transfers across families is arithmetic reachability, and the option-task structure is lineage-private. The habit prior survived its own sealed test on the same sweep, .18, so the second mechanism is not a known-outcome artifact either. So I ran it on a third family. SmolLM3-3B publishes its stage-one checkpoints, and nobody in this program had queried them; its data mix shares the DCLM-derived web corpus (DataComp for Language Models) with OLMo-2 and nothing with the Pile (Pythia's pre-training corpus), its tokenizer splits digit strings differently from either, and it is far stronger at arithmetic by the end of training. Both predictors, tagged where they already sat, at the same bar, each on its own this time rather than one beyond the other; the projection written down before the first checkpoint loaded; twenty-six grid points, the endpoint reproduced byte for byte through a second loader as the gate. Both fired, and both came in above what I had projected. Pythia-1B's counts forecast SmolLM3's order at .170, above the 13B reading; OLMo-2 1B's forecast it at .251, above anything it had ever read on its own lineage. I had projected attenuation for a third family, a new tokenizer and no shared corpus, and none of it showed: two sealed outcomes in a row now say the cross-family forecast does not lose signal crossing families on this battery. The per-task table says what 13B's said, sharpened. The base-8 pair tops both predictors again; the antonym task splits in sign again, negative for the Pile-trained predictor and positive for the DCLM-trained one, so the option-task structure travels with the training corpus rather than the lineage; and reading the four earlier training orders against SmolLM3's, OLMo's two sit far closer than Pythia's two, the corpus overlap visible in the outcome itself. Two of the density findings soften. Three of the four sixty-four-draw blocks clear the bar on this outcome where none did on 13B, and at matched budgets the DCLM-trained predictor leads a third family by the same margin it led its own, so what I had been calling the lineage advantage reads here as corpus plus density. The habit prior fired a third time, at .24, its strongest. That left the corpus itself. Every outcome so far had shared web crawl with at least one predictor; Comma v0.1 is a 7B model trained on one trillion tokens of openly licensed text — papers, forums, code, patents, law, books — with no DCLM-class web at all, so at the source level the overlap inverts: the Pile-trained predictor now shares the outcome's dominant sources and the DCLM-trained one shares almost nothing. Two accounts of the DCLM predictor's margin disagree here and nowhere else — that it came from shared web text, or that it is simply the denser, better-trained read — so I preregistered a third reading beside the two tests: the matched-budget difference between the predictors, with its interval, and whether that interval covers the .066 margin from SmolLM3. Both predictors tagged where they sat, the projection written down with each account's own disconfirmer, twenty-four grid points, the endpoint reproduced byte for byte through a second loader. Both fired: Pythia-1B's counts at .160, OLMo-2 1B's at .210, a fourth family and the first with no web crawl. The margin narrowed to .036, with an interval that excludes the SmolLM3 figure and excludes zero — the DCLM predictor still leads, by less, so the margin moved with the corpus without reversing, and neither account is retired. The outcome-side reading was sharper and went the other way: read against Comma's training order, the three DCLM-class training orders sit closer than the two Pile ones by the same margin as before, and SmolLM3's — the family sharing no source with Comma's Common Pile corpus — is the closest of all. Which items a family learns first does not follow what its corpus shares with the outcome's; it follows something more like recipe and arithmetic strength. The one place the corpus shows is the antonym task: Pythia's sign turns positive on the Pile-like outcome and OLMo's falls, so the split narrows — one task, with the interval still covering zero. The habit prior fired a fourth time, at .23. Three textures from the sweeps are worth keeping. Far more items verify somewhere along training than at its end: on the antonym task, 435 of 500 at 2.8B and 468 at 6.9B are

emittable at some checkpoint against 272 and 286 at the last one, so “when does an item emerge” has no single answer. At 2.8B no task that is flat at the end ever cleared its floor mid-training, but at 6.9B two did and fell back, so a final table can hide transient task-level capabilities. And 6.9B has checkpoints where a task collapses to one constant answer, every one of 500 items answered “13” at a single step on a divisibility-by-13 task and back to normal at the next, which a count over the whole grid absorbs and a “first step at which” outcome would not. OLMo-2’s sweep keeps the transient part of that picture — five flat tasks clear somewhere mid-training and fall back — and shows none of the constant-answer collapses; SmolLM3’s shows fifty-five of them, every one on a task that is flat at the end, and Comma’s seventy-one with a single one on a forecast task, so the pathology is family-specific rather than universal, and a count over the whole grid absorbs it either way.

The last of the predictions I could reach was the third one, that matched loss means matched capabilities regardless of size, and it’s the one where the answer is partly no. Nobody has to train the models: Pythia’s seven sizes were trained on the same data in the same order, and a larger model passes through every loss a smaller one ever reaches, earlier. So for each of the 21 size pairs I bisected the larger model’s public checkpoints to the two adjacent steps where its loss on a fixed two-million-token slice of the Pile’s held-out split straddles the smaller model’s final loss, scored those and the two steps on either side on the 34-task battery, and asked whether the larger model at that loss differs from the smaller model’s final by more than it differs from its own neighbouring checkpoints — the within-size wobble, which on these runs is twelve items per 500 on average. It does, by 27 items per 500 across the 63 cells where any read clears its floor, against a bar of five, with a sign-flip test across the thirteen tasks that carry cells at $p = 6e\text{-}4$, on a rented A100 in 32 hours with every gate clean. What carried it is specific and worth stating without a theory attached. Pythia-2.8B’s final checkpoint is anomalously strong on the arithmetic tasks — 264 of 500 on a three-digit middle-digit subtraction, 92 on base-8 subtraction — and neither 6.9B nor 12B, read at 2.8B’s loss, comes near either number: nine and six, thirty-four and twenty. That one row of the table supplies most of the excess, at 59 items per 500; the other rows sit at ten to eighteen. On the option tasks the six-choice antonym task is ahead for the larger model when the small side is 1B or 1.4B and ahead for the smaller model when the small side is 410M, the same task pointing both ways depending on which final you start from; the plain antonym task never clears cleanly on either side at these losses, hovering at its threshold for every size. Among the 48 cells where the between-size gap exceeds the wobble, 24 favour the larger model and 24 the smaller. That is what a preregistered “no direction” reading looks like cell by cell, and it’s the licence I wrote in advance for it: the excess and the ledger, and no mechanism sentence. Three things sat beside it, and the third is now settled. The experiment was declared underpowered before the first checkpoint loaded — a 17 % chance of detecting the effect I’d written down — and found the effect anyway, which is what the declaration is for. Pythia-12B has a loss spike at step 59,000, a checkpoint where the loss jumps from 1.82 to 2.56 and back in a thousand steps and the model briefly forgets most of the battery; my search rule, which bisects to any adjacent pair straddling the target loss, landed one of the 21 brackets on that spike’s edge, and the verdict is unchanged with that pair removed. And the smaller model’s final was itself one read of a moving quantity — the last thousand steps of Pythia’s anneal move a rising task’s count by up to 75 items — so a design that averages the larger side’s wobble and takes the smaller side’s final at face value had measured only half the noise. So I measured the other half: each smaller model read at its three preceding checkpoints as well, on a second rented A100 that reproduced every one of the first campaign’s reads byte for byte, with both sides’ wobble subtracted by a statistic whose false-positive rate I had computed in advance on a grid of small-side noise so that it could neither invent an excess nor

erase one. The smaller side's wobble turned out to be the smaller of the two, nine items per 500 against the larger side's twelve, and the excess net of both is 27.6 items per 500, p = 9e-4, 15 of them outside the 2.8B row; the un-averaged read had cost the first reading nothing, its false-positive rate at the wobble actually measured being a tenth of the nominal one. What I get to keep is narrow and I'd rather keep it narrow. At equal loss on this battery the profiles are not the same, the difference is dominated by one final checkpoint that is unusually good at arithmetic, and nothing in it says bigger models get capabilities that loss-matched smaller ones lack. And those same sixty checkpoint units let me test the original parameter-axis claim directly, with training data held completely constant: at the final checkpoint, where every Pythia size has seen the exact same 300 billion tokens in the exact same order, 12B beats 410M by 4.16 percentage points across the 34-task battery (with 2.8B peaking in mean accuracy and 6.9B and 12B clearing the most tasks), and the 1B sampled output tail orders which individual items resolve across the four larger parameter sizes — 1.4B, 2.8B, 6.9B and 12B, with 410M and 1B excluded from the outcome — at .218 for Pythia-1B (256 draws, p = 1e-4) and .278 for cross-family OLMo-2 1B (64 draws, p = 1e-4, positive on all nine tasks), while the 1B hidden-state linear probe sits flat at .026 (p = .13). Restricting strictly to items that neither 410M nor 1B can solve under greedy decoding at 300 billion tokens leaves the parameter-ladder ordering intact (.194 for Pythia-1B and .270 for OLMo-2 1B, both p = 1e-4); scoring each item by the smallest parameter count from 1.4B to 12B that first solves it gives .198 and .263; and dropping the 2.8B arithmetic outlier raises the parameter-ladder concordance to .237 and .306. Even when training tokens are held fixed, what a 1B model sees dimly in its sampled tail is what adding parameters brings into greedy focus item by item.

So here's the scoreboard, stated at the resolution it was bought and read against the thesis as I now hold it: the structure is in the corpus, a small model holds a blurred view of it, and what comes into focus first is forecastable from the blur, by the tail of the model's own outputs and not by a probe on its hidden state. On synthetic ground truth, the probe leg of the discriminator separates structure-present from structure-absent systems once the untrained floor is subtracted, and nothing decodable accumulates below a percolation threshold at a resolution that rules out accumulation above 39% of the super-critical margin. On one real model family, at 410M and 1B, on two task classes, the three instruments read in one order and never the other: representation first, deep sampling second, greedy last, eight cells without an inversion. The rare sampled successes under a greedy zero are forecastable at item grain from outside the model, and they aren't a copying artifact. The cross-task forecast from 1B to 2.8B and beyond, the prediction this framing makes that the phase-transition view doesn't, hasn't been found on a 34-task battery by any instrument I've tried, with the blind region stated each time. The eight sealed forecasts I ran — item order inside 2.8B's, 6.9B's, OLMo-2 7B's, OLMo-2 13B's, SmolLM3-3B's and then Comma v0.1's training, the last two from two predictors at once, each hidden from everyone until its sweep — were carried by the sampled channel at .17, .20, .22, .18, .17, .25, .16 and .21 and by the probe not at all where it competed, at a power that makes the zeros measurements rather than shrugs. The cross-family predictor stopped short of its preregistered bar on 7B at sixty-four draws, .095, cleared the same bar on that already-known outcome at two hundred and fifty-six, .155, and then — the test that matters — forecast 13B's genuinely sealed order at .126 above the bar, the program's first cross-family forecast, with the four sixty-four-draw blocks all under the bar on that same outcome, and then SmolLM3-3B's at .170 and Comma v0.1's at .160 with no attenuation at all: the shortfall was predictor density rather than a family wall, and the wall that remains is task-shaped — arithmetic transfers across families, and the antonym task is the one place the corpus shows, disagreeing in sign between the two predictors on the DCLM-class outcomes and narrowing on the Pile-like one. The DCLM-trained predictor stays ahead at any budget, on its own lineage, on the third family and, by

less, on a family trained on none of its web; which training orders resemble which does not follow the corpus at all. Three fifths of the within-lineage forecast survives holding answer length, repetition, prompt overlap and the small model's answer habits fixed; the other two fifths are length and habit on one task, and the habit forecasts by itself. The lens claim the distribution-level story leans on is still a citation. I measured it at task grain on fifteen tasks across four families' training runs and got .61 of the task-specific agreement in place before the task first performed, on an outcome already known; then I ran the tasks that never perform through the same selection, every input known to me in advance, and they produce that number a quarter of the time. The lead over them is .17 with an interval covering zero, no single run separates, and what stands is only that the agreement didn't arrive with performance. A sealed replication on two more runs, with a rank statistic whose null owes nothing to the data, found the rising tasks at the flat pool's median, U .45, the two runs disagreeing in sign; the lens claim is a citation with two disclosures beside it now. At equal held-out loss, Pythia's larger models do not have the smaller models' final profiles on this battery — an excess of 27.6 items per 500 over both sides' wobble across 63 cells, p = 9e-4, with no direction: 26 cells favour the larger model and 24 the smaller, most of the excess comes from one final checkpoint that is unusually good at arithmetic, and the follow-up that read the smaller side's wobble found it the smaller of the two. Nothing here touches a frontier model, nothing reads from above 1B on the predictor side, and everything runs on Pythia, OLMo-2, SmolLM3 and Comma. I'd rather say that plainly than let the reversal result carry more than it can — it's the strongest thing I have, and it's a toy.

That was the scoreboard on the first battery. The next test took the forecast off it. I regenerated seventeen task difficulty tiers ("rungs") drawn from ten benchmark-defined tasks, fresh items under renderings fixed in advance, and sampled the two small models before any outcome sweep. Then I scored 87 training checkpoints across the four families. Every one of the eight forecasts cleared the preregistered .10 effect bar: Pythia at .154, .143, .112 and .169; OLMo at .194, .207, .158 and .184, in SmolLM3, OLMo-7B, OLMo-13B and Comma order. Every p is 1/10,001, the minimum ten thousand permutations can report. The tests share items and predictors; eight passes aren't eight independent replications. Strings and choices each carry positive per-rung intervals on at least three outcomes for both predictors. The checks that condition on more item structure and retain only rungs that beat the frozen heuristics keep all eight forecasts above the bar, descriptively. This is now the strongest forecast result in the program, and the old battery's arithmetic-versus-options boundary doesn't describe it.

There are bounds on that reading. Comma's long-unscrambling rung supplied only thirteen ever-correct items, below the twenty required, so its two tests read nine rungs where their power simulations had read ten; the other six tests read exactly their power sets. Four early OLMo-13B checkpoints produced non-finite probed logits and were retained as emitted under the rule frozen before collection. These aren't BIG-bench's original scores: the items are fresh, the rendering departures are disclosed, and familiarity with the task formats is still possible. Nothing here forecasts across tasks, establishes a learning mechanism, or touches frontier scale. What changed is specific: the item-order forecast holds beyond the battery it was found on, for both small predictors on every named outcome.

I ran one more test to see what that blur at 1B actually resolves: whether the small model is already weakly tracking the fine input-to-answer rule, or only resolving the coarser outline of which answers belong to the task while the fine detail comes into focus later. I built 2,000 counterfactual prompt pairs across four families from the second battery (five-entity deduction, polygon shapes, unit conversion and word unscrambling), each pair swapping one input detail to flip the target answer from one string to its

counterfactual partner alongside sham rewordings of each that leave both targets intact (four prompt variants, or "arms," per pair — 8,000 prompts in all). I sealed 64 draws per prompt from Pythia-1B and 64 from OLMo-2 1B (plus 16 from the untrained twin, 1.152 million draws in all), scored how selectively each 1B model shifts its conditional mass between the pair's two targets when the prompt changes, and swept SmolLM3-3B's twenty-six checkpoints, requiring all four arms of a pair to verify at the same checkpoint and at least three families to clear twenty joint-positive pairs. The formal test came back undetermined on both predictors because only unit conversion cleared the twenty-pair floor, at 50 of 500 pairs; unscrambling gave 2, and deduction and shapes gave 0. Under the preregistered licence, that means there aren't enough informative families for the joint forecast rule, shared input-sensitive difficulty and learned heuristics stay live explanations, and the test doesn't identify an internal algorithm or separate resolution from new computation on its own. What the numbers inside the run show is coarse-to-fine resolution in stages. On shapes, SmolLM3-3B solves about a quarter of the individual prompts on each arm (120 to 125 of 500) yet scores 0 of 500 on four-arm pairs: at 3B it has resolved the four-word polygon class well enough to pick a checkpoint-wide favorite, which a single-item benchmark credits as emerging capability, without yet resolving the fine coordinate detail that flips one polygon into its partner. At 1B, mean conditional target selectivity sits near zero in every cell, from −.006 to +.024 against a .05 bar, so the 1B blur isn't a miniature version of the counterfactual rule. And on unit conversion, where 3B finally resolves both halves of 50 pairs, 1B conditional target contrast (whether 1B shifts probability toward the *right* target when the prompt detail flips) orders 3B's joint emergence negatively (−.162 for Pythia-1B and −.093 for OLMo-2 1B), while unconditional target-pair availability (how often 1B emits *either* of the pair's two target answers on that pair's prompts, .141 and .186) and wrong-target habit rate (how often 1B emits those same target numbers as wrong answers on *other* prompts, .200 and .198) still forecast it at the old magnitudes. At 1B the model resolves the coarse layer, which target numbers belong to the prompt format and sit in its active output vocabulary, and those are the first pairs where 3B later resolves the fine counterfactual detail.

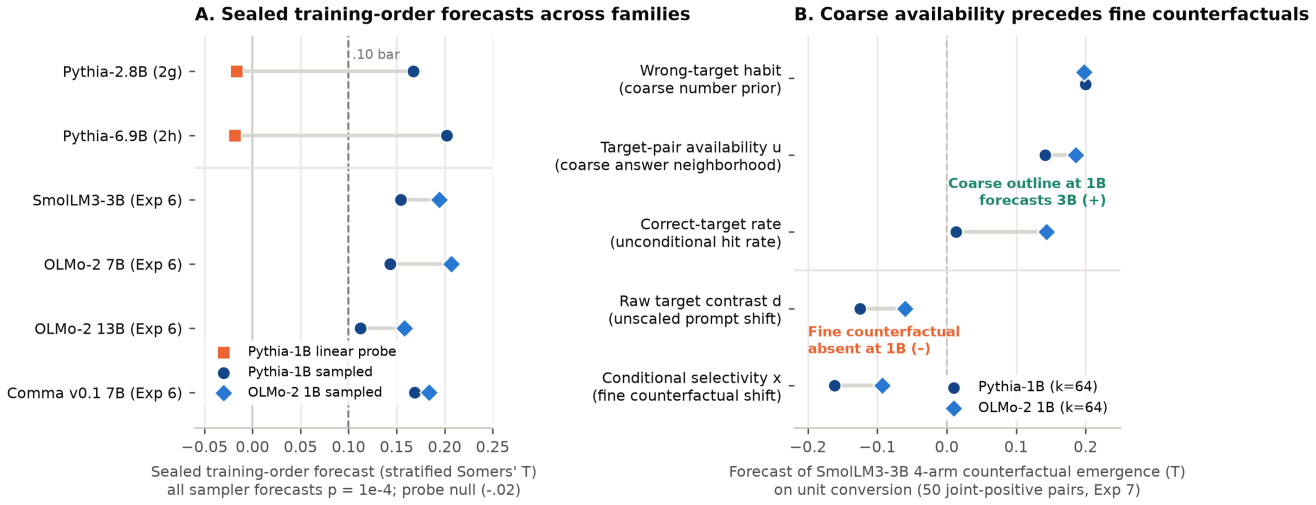


Figure 2. Sealed training-order forecasts across four model families and coarse-to-fine resolution inside the 1B blur. A: Stratified rank concordance (Somers' T, ranging from −1 to +1 with 0 at chance) forecasting which items become greedy-emittable first during training on Pythia-2.8B and 6.9B (Experiments 2g and 2h, where the 1B sampled tail scores +.170 and +.202 while the 1B linear probe sits at -.019) and across all eight sealed tests on fresh benchmark-derived tasks in SmolLM3-3B, OLMo-2 7B,

OLMo-2 13B and Comma v0.1 7B (Experiment 6, all eight clearing the .10 bar at p = 1e-4). B: Decomposition on unit conversion in Experiment 7 (50 four-arm counterfactual pairs resolved by SmolLM3-3B): 1B coarse target-neighborhood availability (T_u = +.141 and +.186) and wrong-target number habit (T_wrong = +.200 and +.198) forecast which counterfactual pairs 3B resolves first, whereas 1B fine prompt-conditional target shifting is absent (T_x = -.162 and -.093).

But What If the Resolution View Is Wrong?

The framing I've laid out should earn its keep, so let me engage with the strongest version of the opposing case.

The genuine-emergence camp — and there are smart people in it, including some of the co-authors of the original scaling laws papers — would argue that I'm conflating two things: *representation* and *capability*. Of course a 7B model's weights encode statistical regularities that correlate with reasoning; the model has seen billions of tokens of human text. But being able to *represent* the structure of a reasoning chain in your hidden states is not the same as being able to *execute* it. A small model might have the first step encoded with high fidelity, but the second step depends on the output of the first, and the error propagation means it can't actually *perform* the chain even in principle — the attractor basin for the correct path is too narrow to be reachable from the model's own sampled outputs, even if a probe finds it in the activations.

And this camp now has a formal model. Lubana et al. (2024) trained transformers on a context-sensitive formal language and showed that the sudden acquisition of capabilities can be modeled as *percolation on a bipartite graph* — a literal phase transition, with the machinery to predict where the transition point moves when you change the data structure. In their setting, the capability genuinely doesn't exist below the threshold: the network hasn't yet acquired the connected structure that the capability runs on. That's not a measurement artifact. If frontier-model emergence works the way their formal-language emergence works, the resolution framing is wrong about at least some capabilities.

There's real bite to this. The resolution model doesn't claim that every latent structure is reachable at every scale; an attractor basin narrower than the model's typical sampling variance is effectively unreachable, which is why the 4-bit cat is unrecognizable. And I'll concede the percolation result carves out a class of capabilities, ones that depend on acquiring a connected *structure* rather than amplifying an existing signal, where "it was there all along" is simply false. The question is which class the capabilities we care about fall into. Here the published record leans toward resolution: for the canonical emergent abilities (multi-step reasoning, in-context learning, instruction following), the literature finds probeable structure in small models, elicits it by sampling, and forecasts its arrival from below, and a capability that percolates into existence should show none of those signatures. But my own runs forced a correction to how the test has to be stated. The three signatures aren't three views of one fact; they're rungs on an instrument ladder ordered by power. Probes see deepest. Exhaustive sampling sees next, with a floor set by the budget: 128,000 draws resolves per-draw rates down to about 1e-5 and nothing below, and the first thing a deeper look found was a channel the shallower one had called absent. Argmax sees last. So the discriminator isn't "all three present" versus "all three absent"; it's whether the signatures appear in ladder order as instrument power increases. Percolation-class capabilities should show nothing at any rung, probes included. That's what Lubana-style networks show below threshold, in their paper and in my replication of it, and it's what keeps the carve-out testable rather than rhetorical. One caution from the battery tests: instrument ladder order within a task

(probe, then deep sampling, then greedy) is a different question from which 1B signal forecasts what resolves at larger scale. On reversal and on arithmetic, the linear probe reads the answer before sampling or greedy decoding can emit it; when forecasting what larger models resolve next, however, the leading indicator is the 1B model's sampled output tail rather than the linear probe, and it forecasts resolution order at item grain — across both parameter scale at fixed training tokens and training checkpoints across five model families — rather than as a 34-dot task-average ranking across incompatible task formats.

I think the resolution framing earns its keep on two grounds. First, it's consistent with the smooth log-probability curves, the probe-ability of hidden states, the PassUntil and test-time-compute results, and the loss-threshold findings, while strict phase-transition models need additional assumptions to explain each of those. Second, it suggests more productive research directions — and it makes predictions the other view doesn't, which is where I want to go next.

What the Resolution View Predicts

A framing that can't be wrong isn't worth much. Here are five predictions the resolution model makes that a strict phase-transition model doesn't, each checkable with current techniques.

1. No capability arrives without a precursor signal. For any capability that "emerges" at scale N under a binary metric, models meaningfully below N should show a nonzero, smoothly increasing pass rate under exhaustive sampling (PassUntil-style) and above-chance linear-probe accuracy on the relevant intermediate quantities. The phase-transition view permits — indeed expects — capabilities whose pass rate is genuinely zero below threshold. Every capability that arrives at scale with *no* precursor signal in smaller models is a strike against the resolution view. To my knowledge, no headline capability has yet been shown to arrive signal-free; I'd love to see someone try to produce one. One refinement from my own runs: "nonzero pass rate under exhaustive sampling" has to carry its budget with it. A rate of 1e-5 is invisible at a thousand draws and obvious at a million; below the sampling floor there's a regime where only the probe arm of this prediction has teeth. I've now watched this happen under preregistration: the same cell read zero at 128,000 draws and a measured 6e-6 at 512,000, so the zero was a fact about the budget, not the model.

2. Probes forecast the capability ladder. Rank capabilities by linear-probe accuracy in today's models at scale N. That ranking should predict the order in which those capabilities become reliable at argmax in models at scale 2N, 4N, and so on. The phase-transition view gives you no reason to expect probe accuracy — a fact about representation — to forecast execution. The resolution view says representation *is* the leading indicator, because execution is just representation plus enough SNR to survive decoding.

This is the prediction I've tested hardest, and the runs forced a sharp correction to *where* the leading indicator lives and *at what grain* it operates. In its original form — collapsing a 34-task Pythia battery into 34 task-average dots subtracted against format-guessing floors, and asking a linear probe at 410M/1B to rank which tasks clear their floor by 12B — it failed across three tries, because 23 of the 34 tasks sit tied at zero at 1B, different tasks carry incommensurable answer spaces, and linear probes read static token features rather than the decoding bottleneck. On the two tasks I went back to at item grain, the probe's earlier silence was the label I'd given it, and the representation reads the answer at 1B once asked for the matched label. The checkpoint experiments then pitted the matched-label linear probe directly against the 1B model's own sampled output tail under sealed preregistration: against the order in which Pythia-2.8B's

and 6.9B's training makes individual items greedy-emittable, the 1B linear probe forecast nothing (concordance –.02 against .95 power to detect .15), while the 1B sampled output tail forecast the emergence order at .17 and .20 through the same difficulty controls. Prediction 2 as originally written said hidden-state linear decodability is the leading indicator; on every test where the two competed, the leading indicator is the tail of the small model's output distribution. And once read in the 1B sampled output tail at item grain, the forecast holds across both axes of scale. Along the **parameter axis at fixed training data** (holding all Pythia models at `step143000`, 300 billion tokens), the 1B sampled tail predicts which individual items resolve across 1.4B, 2.8B, 6.9B and 12B at .218 for Pythia-1B (256 draws) and .278 for cross-family OLMo-2 1B (64 draws, both p = 1e-4, and .286 unstratified across all 29 informative rungs), while the 1B linear probe sits at .026. Along the **training axis across model families**, sealed 1B output-tail forecasts cleared the preregistered bar on OLMo-2 7B (.215 within-lineage), OLMo-2 13B (.126 from Pythia-1B at 256 draws and .18 from OLMo-2 1B), SmolLM3-3B (.170 and .251), and Comma v0.1 7B (.160 and .210), with sixty percent of the within-lineage forecast surviving after conditioning simultaneously on answer length, character repetition, prompt overlap and the small model's wrong-answer habits. Taken off the original synthetic battery onto seventeen fresh benchmark-derived rungs across ten tasks (Experiment 6), **all eight sealed primary tests** (Pythia-1B and OLMo-2 1B across SmolLM3-3B, OLMo-2 7B, OLMo-2 13B and Comma v0.1 7B) cleared the bar (.112 to .207, each p = 1/10,001). Finally, decomposing that signal on paired counterfactual prompts (Experiment 7) showed why it works: at 1B, the output tail resolves the *coarse* layer first — which target strings belong to the task's output neighborhood (.141 and .186) and number-habit prior (.200 and .198) — and those are the pairs where 3B first resolves the *fine* counterfactual input-to-answer rule.

3. Matched loss means matched capabilities, regardless of size. If fidelity, not parameter count, is the operative variable, and pre-training loss reads fidelity faithfully, then models trained to the same loss on the same distribution should show the same capability profile even at very different sizes — including on tasks with "emergent" reputations. The second condition is the one I hadn't written down in the first version of this essay. It's doing the work in the prediction, and it's the one that gave. Du et al. (2024) have already confirmed this in a controlled setting; the prediction is that it continues to hold as the comparison is pushed to wider size ratios and harder capabilities. Finding a capability that a 70B model has and a loss-matched 7B model lacks would be genuine evidence for size-dependent emergence. I ran the version of this I could afford, on Pythia's public checkpoints rather than models trained for the purpose: seven sizes from 160M to 12B, every larger model read at the checkpoint where its held-out loss equals a smaller model's final loss, 21 pairs at ratios to 73×, the same 34-task battery, the comparison scored against the larger model's own wobble between neighbouring checkpoints. The profiles differ by more than that wobble — an excess of 27.6 items per 500 averaged over the 63 live cells once both sides' wobble is subtracted, against a preregistered bar of five, at p = 9e-4 under an exact sign-flip test across tasks — and the excess has no direction the design can read: among the cells where it shows, the larger model is ahead in about as many as the smaller one is. So loss is not the whole currency on this battery, and the part it isn't doesn't point toward size. That is the preregistered reading and I'm not entitled to more from it: the first run was declared underpowered before it ran and the follow-up that measured the smaller side's wobble was powered and found the same excess, it's one family, one synthetic battery and sizes to 12B, and I say below what carried the excess. For the thesis it's the second of the three revisions stated up front: fidelity is the knob, parameters and training both turn it, and loss reads it imperfectly.

4. Test-time compute and scale are partially interchangeable — with a specific exchange rate. For latent-but-noisy capabilities, inference compute (sampling, search, verification) should substitute for training scale, and the substitution should be governed by the small model's pass rate: wherever PassUntil finds a non-trivial rate, test-time compute closes the gap to larger models. Where the pass rate is genuinely zero, no amount of test-time compute helps. Snell et al. (2024) confirmed the first half; the second half is the falsifiable edge, because it marks the boundary between resolution-style capabilities and percolation-style ones.

5. SNR interventions shift thresholds without adding parameters. Anything that raises representational fidelity per parameter — cleaner data, longer training, better tokenization, higher-precision arithmetic, distillation from a sharper teacher — should move "emergence" thresholds earlier on the parameter axis. The phase-transition view ties the threshold to scale; the resolution view ties it to fidelity, of which scale is one input. The steady march of capability into ever-smaller models over the past two years is consistent with this, but a controlled demonstration — same architecture, same parameter count, threshold moved solely by fidelity interventions — would be the clean test.

And to be symmetric about it, here's what would falsify the resolution view: a replicable demonstration of a frontier-relevant capability that appears at scale with zero precursor pass rate, zero probe signal, and no forecastability from below — a genuine percolation event in the wild. If Lubana-style transitions turn out to be the norm for the capabilities that matter rather than a special case of structured synthetic tasks, this essay is wrong.

The Shape of the Threshold

Here's a useful way to think about it. Different tasks require different amounts of representational fidelity. Simple fact recall might resolve at the equivalent of "2-bit" resolution — easy for even a tiny model. Multi-step arithmetic might require "8-bit" resolution. Composing multiple reasoning steps while tracking long-range context might need "32-bit."

This explains the "capability ladder" — the observation that capabilities and task instances appear to emerge in a consistent order as models scale. They're not appearing in order of *discovery*. They're appearing in order of *resolution requirement*. The simplest, coarsest structures cross the SNR threshold first. The finest, most demanding ones cross last. (Prediction 2 above is this claim made quantitative: once read in the 1B model's sampled output tail rather than a linear probe, it forecasts which items cross the threshold first both as parameter count increases at fixed training data and as training progresses across five model families, while coarse target availability at 1B precedes fine counterfactual switching at 3B.)

It also explains *grokking* — that bizarre phenomenon where a model memorizes training data for thousands of steps, then suddenly generalizes. Under the resolution model, early training allocates representational bandwidth to memorization because it gets the fastest loss reduction. The latent structure is there, but it's being outcompeted. Continued optimization gradually compresses the representations, sharpens the features, and at some point the bandwidth allocation flips — the structured solution wins because it's more compressible. The capability appears to emerge from nowhere. In reality, it won a resource allocation battle.

So What?

If this framing is correct, it points in a few interesting directions.

Scale isn't the only knob. Suppose resolution scales roughly logarithmically with parameters — each doubling of parameters buys you a fixed increment of representational fidelity. I'm speculating about the exact shape here; nobody has measured a "bits per parameter" curve for these structures, and the true relationship is probably task-dependent. But if anything like it holds, raw scale is an expensive way to improve. Higher-precision training, mixture-of-experts architectures with fine-grained routing, techniques that effectively increase representational capacity without adding parameters — all of these become more attractive. And the test-time compute results have already demonstrated the inference-side version of this trade: you might get more resolution per dollar by decoding a smaller model harder than by training a larger model conventionally.

The right question shifts. The field has been asking "how big does a model need to be before capability X appears?" The resolution framing suggests a different question: "what's the most efficient way to increase representational SNR?" That's a much more actionable research direction. The first version of this essay said the answer would be denominated in pre-training loss rather than parameters. On the one battery where I checked, loss-matched models don't carry matched profiles, so loss is the better of the two gauges and still not the quantity itself. Whatever fidelity is denominated in, it isn't parameters, and it isn't quite loss either.

Safety implications change. If capabilities are latent in small models, the unpredictability concern isn't about new things appearing — it's about the *noise floor dropping* below previously unfathomable capabilities. A model that could only express a dangerous capability 0.1% of the time at 7B parameters might express it 90% of the time at 70B. The capability didn't emerge. It just got loud enough to hear.

What would this mean in practice for safety teams? Three concrete things. First, any red-teaming protocol that evaluates a model only on its argmax outputs (what it actually produces at temperature 0) is systematically blind to latent risks. Evaluations should routinely include probing for capabilities below the decode threshold — linear probes on hidden states, log-probability analysis on harmful completions, and pass-rate estimation via repeated sampling at higher temperatures. Second, scaling decisions should be informed by latent-capability baselines: if a 7B model already shows a 0.1% pass rate on a dangerous behavior, a 70B model should be expected to show it reliably, and safety mitigations should be designed with that expectation rather than treated as a surprise. Third — and this is where the framing cuts against a popular hope — the unlearning results say that any mitigation applied at decode time is shallow by construction. Refusal training, unlearning, suppression: all of them lower the gain on a signal that's still on the wire, and anything that restores a little gain (a small fine-tune, quantization, a clever prompt, test-time search) brings it back. The intervention that operates at the right level of the channel is upstream: what goes into the training distribution. If you don't want the capability in the photograph, the durable options are to treat the sore or not photograph it. Retouching loses to anyone with the negative.

One more safety wrinkle the recent literature adds: test-time compute means the noise floor is no longer fixed at deployment. A capability that's safely below the decode threshold at temperature 0 may be reachable by anyone willing to spend inference compute on search and verification. Latent-capability evaluation isn't just a training-time concern anymore; it's an every-deployment concern.

A Substitution Test

The next time you hear someone say "this capability emerged at 70B parameters," try substituting "was resolvable at 70B parameters." Does the sentence still hold? Most of the time, it does. The capability was latent in the smaller model — quantized into too few bits to be useful, competing for bandwidth with simpler patterns, but structurally present.

The signal was already on the wire — and the wire runs all the way back to the data. Scaling didn't create the structure; scaling lifted it above the noise floor. The benchmark jump everyone calls emergence is the moment the receiver locks on, and the field has been reading that moment backwards. The capability was the cause. The jump was the effect. Mistaking the detector's click for the particle's creation is the inversion this essay is trying to undo — because everything downstream follows from it: the surprise at each new model, the retouched-photograph safety methods, the mystification of scale.

Emergent abilities are real — but not in the way the term suggests. They're not gifts from scaling laws. They're latent quantities we finally have the resolution to read.

Further Reading

The papers mentioned in this piece, annotated:

The original debate:

- **Belrose et al. (2023). "Eliciting Latent Predictions from Transformers with the Tuned Lens."** Decodes each intermediate layer directly, showing refined predictions forming well before the output layer — the cleanest window into models encoding more than they surface. If you read one paper from this list, read this one.
- **Schaeffer, Miranda, & Koyejo (2023). "Are Emergent Abilities of Large Language Models a Mirage?"** The paper that kicked off the metric-artifact debate. An accessible read that pairs well with the counterarguments in Wei et al.'s response.
- **Wei et al. (2022). "Emergent Abilities of Large Language Models."** The canonical statement of the emergence-as-phase-transition view. Worth reading alongside Schaeffer to see both sides of the argument directly from their strongest proponents.
- **Olsson et al. (2022). "In-context Learning and Induction Heads."** The mechanistic evidence that the circuit underlying in-context learning exists at very small scales, even when it doesn't reliably fire.
- **Hu et al. (2023). "Predicting Emergent Abilities with Infinite Resolution Evaluation."** The PassUntil technique — empirical demonstration that small models can produce correct reasoning chains with sufficient sampling. Published at ICLR 2024; arXiv:2310.03262.
- **Nanda et al. (2023). "Progress Measures for Grokking via Mechanistic Interpretability."** Provides a complementary mechanistic picture of how latent structure gradually wins out over memorization in grokking, which aligns with the resolution model's bandwidth-allocation explanation.

The follow-up literature:

- **Du et al. (2024). "Understanding Emergent Abilities of Language Models from the Loss Perspective."** NeurIPS 2024; arXiv:2403.15796. Emergence thresholds live in pre-training loss, not parameter count — the strongest evidence that fidelity, not size, is the operative variable.
- **Ruan, Maddison, & Hashimoto (2024). "Observational Scaling Laws and the Predictability of Language Model Performance."** NeurIPS 2024 spotlight; arXiv:2405.10938. Emergent phenomena follow smooth sigmoids predictable from small models, using only publicly available checkpoints.
- **Schaeffer et al. (2024). "Why Has Predicting Downstream Capabilities of Frontier AI Models with Scale Remained Elusive?"** ICML 2025; arXiv:2406.04391. Traces exactly where the evaluation pipeline destroys the statistical link between scale and measured performance.
- **Snell et al. (2024). "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters."** arXiv:2408.03314. Inference compute substitutes for training scale wherever the small model's pass rate is non-trivial — the engineering counterpart of PassUntil.
- **Lubana et al. (2024). "A Percolation Model of Emergence: Analyzing Transformers Trained on a Formal Language."** arXiv:2408.12578. The strongest current case for genuine phase transitions — read it as the boundary of the resolution framing's territory, and as the source of this essay's falsification criterion.
- **Berti, Giorgi, & Kasneci (2025). "Emergent Abilities in Large Language Models: A Survey."** arXiv:2503.05788. A field map of the whole debate, including its extension to reasoning models and inference-time search.
- **Huh et al. (2024). "The Platonic Representation Hypothesis."** ICML 2024; arXiv:2405.07987. Model representations converge across architectures and modalities as scale increases — many lenses, one scene.
- **Łucki et al. (2024). "An Adversarial Perspective on Machine Unlearning for AI Safety."** arXiv:2409.18025. Fine-tuning on ten unrelated examples recovers capabilities "removed" by state-of-the-art unlearning — suppression operates at decode time, not on the structure.
- **Zhang et al. (2024). "Catastrophic Failure of LLM Unlearning via Quantization."** arXiv:2410.16454. Quantizing an unlearned model restores most of the forgotten knowledge with no adversary at all.

All are freely available on arXiv or in open-access proceedings.